

My Fixes Became the Lesson: Teaching AI Better Coding Habits

François Martin

Hands up: which agent?

Claude Code

Codex

Copilot, Cursor, Junie,
or another agent

No coding agent yet

The worst code is not the code that is clearly wrong.

It is the code that is almost right.

Coding agents are painfully good at almost right.

Stack Overflow Developer Survey 2025

66%

name "AI solutions that are almost right, but not quite", the most common frustration

Curriculum Vitae



Skills: writes code almost right

Yet the industry replaces humans with AI
in a heartbeat.

May 2026

ALMOST RIGHT

```
Optional<Comment> hit = comments.stream()
        .filter(this::isWorkpad).findFirst();
if (hit.isPresent()) {
    return update(hit.get(), text);
}
return create(card, text);
```

> ### My prompt
>
> Get rid of the `Optional` antipatterns. Search for `isPresent` in the codebase. In most cases where `isPresent` is followed by `get`, there is a better way of doing it.

Round 1

ALMOST RIGHT

```
Comment workpad = comments.stream()
        .filter(this::isWorkpad).findFirst()
        .orElse(null);
if (workpad != null) return update(workpad, text);
return create(card, text);
```

> ### My prompt
>
> It used `orElse(null)`, which is equally smelly as `isPresent`. With the `if (... != null)` after it, this is effectively the same thing. Add that to `AGENTS.md`.

Round 2

ALMOST RIGHT

> ### Codex, in `AGENTS.md`
>
> Do not use `isPresent()` followed by `get()` or `orElseThrow()`.
>
> Do not use `orElse(null)` just to branch on `!= null`.

```
Optional<String> id = options.workspaceId();
if (id.isPresent()) return id.orElseThrow();
return askForWorkspace(terminal);
```

> ### My prompt
>
> The PR adds this guardrail to `AGENTS.md`. However, it still introduces or preserves the same antipattern in touched code.

Round 3

ALMOST RIGHT

```
for (String id : options.workspaceId()
        .stream().toList()) {
    return id;
}
return askForWorkspace(terminal);
```

> ### My prompt
>
> I feel like it cheated by doing `stream().toList()` on `Optional` values. Isn't there a way to solve it properly?

Round 4

ALMOST RIGHT

```
return OptionalIo.mapOrElseGet(
        options.workspaceId(),
        id -> id,
        () -> askForWorkspace(terminal)
);
```

> **My prompt**
>
> Is this really the best practice solution?

Every round compiled.

Every round worked.

None was the code I wanted.

Why I did not just fix it myself

I say what is wrong,
not what to write.

That leaves it room.
Sometimes it surprises me.

I wanted to find its limits.

By hand was faster.
Speed was not the question.

"I need to make sure I never have to go through this again."

I needed the agent to be more skilled.

Wait. Skill.

Hands up

You corrected an agent twice for the same thing

You have used a skill someone else wrote

You have built one yourself

You were happy with how it came out

What is a skill?

An open format,
read by 40+ tools

A folder the agent reads
when it matters

One Markdown file
with a name and a description

Extra files it opens
only when needed

The description is the trigger

Read at startup, for every installed skill.

```
name: java-optionals
description: Write, review, and refactor Java
  Optional code ... Use whenever writing,
  reviewing, or refactoring Java code that
  reasons about Optional ...
```

The real header of `java-optionals/SKILL.md`, shortened

Or call it by name: `/java-optionals` in Claude Code
`$java-optionals` in Codex

Progressive disclosure

Tokens loaded for one prompt	`java-optionals.md`	`java-optionals/` `SKILL.md` `references/` `examples.md`
Every prompt	8,457	117
Touches `Optional`	8,457	994 (+877)
Needs the examples file	8,457	6,085 (+5,091)

Capture

Once the agent did what you wanted,
in the same conversation:



> **My prompt**
>
> Check the transcript of this conversation for what was bad and what was good. Use that to create a skill for `Optional` code in Java. Don't add things speculatively: base it on what you implemented badly without my guidance.

Inside the skill it wrote

Bad attempted fix:

```
Comment w = found.orElse(null);
if (w != null) { return update(w, text); }
return create(card, text);
```

Better final shape:

```
return found.map(w -> update(w, text))
            .orElseGet(() -> create(card, text));
```

Replay the first prompt

Check out the commit
before your first prompt.

Install the skill.

Type the same prompt.

Not what you expected?
The skill is not done.

Problem: next month

A new model next month:
is the skill still needed?

You improve the skill:
did you break anything?

A commit hash and a prompt
to remember, forever

Another team cannot run it
without your code

Solution: a minimal reproducible example

```
Create RetryBackoff.java. Assume Java 17.

If the response has a valid Retry-After header,
return that duration. Otherwise return the
exponential backoff. Include
Optional<Duration> parseRetryAfter(String value).
```

The task does not hint at the answer.

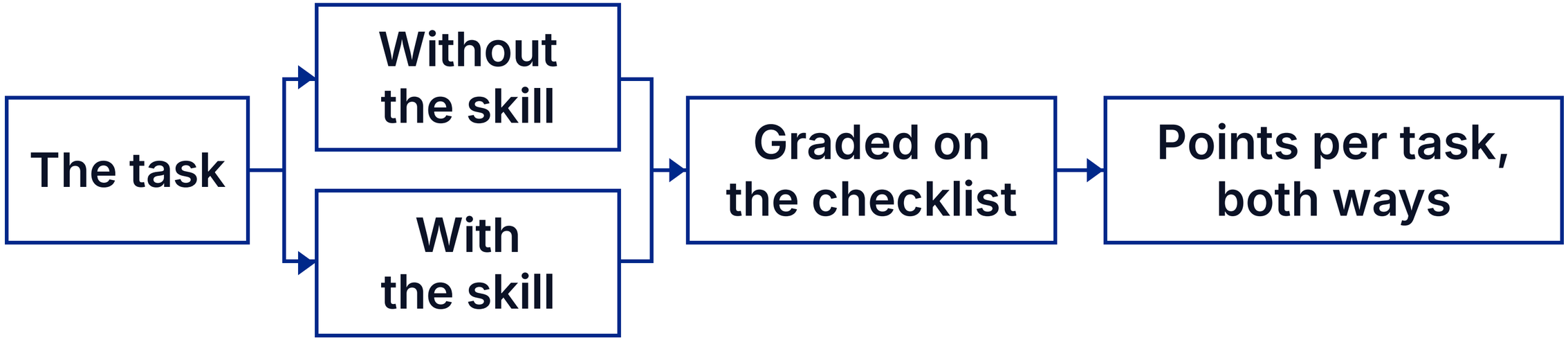
How the answer is graded

A checklist scores the answer, up to a total of 100 points.

60	An `Optional`, read with `flatMap` and `orElseGet`
20	No antipatterns
20	Behavior and readability

A runner that does exactly this

tessl: evals for skills, with and without.



Say what done looks like

> **My prompt**
>
> Tweak the skill until it is at 100 percent in all evals with the skill. However, don't cheat and don't be leading. Don't make the skill so specific that it literally tells it what to do in that example.

Four tasks, without and with

task	without	with
retry backoff	19%	100%
command sanitizer	41%	100%
port lookup	55%	100%
checked exceptions	73%	100%
all four	44%	100%

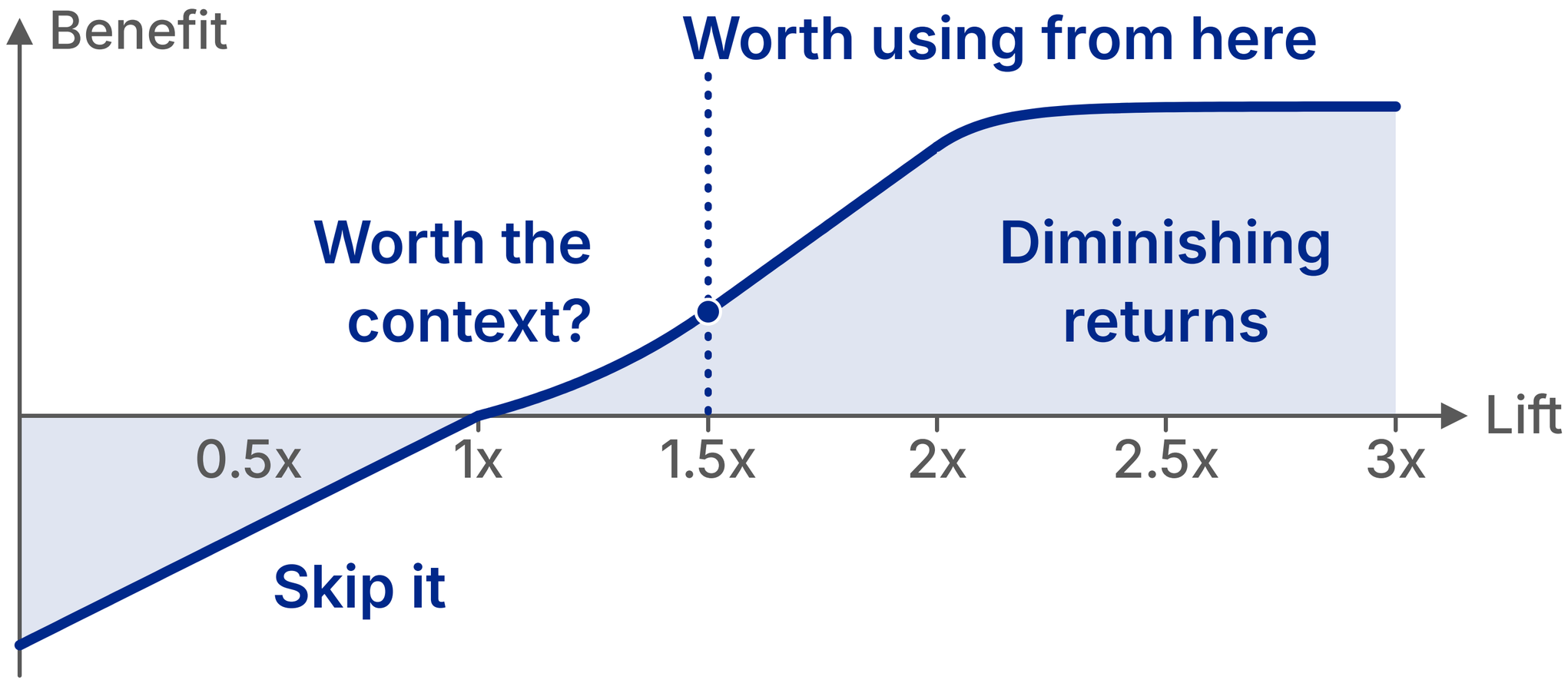
java-optionals evals, Sonnet 4.6 as the solver, June 2026

Lift

$$\frac{\text{points with the skill}}{\text{points without}} = \frac{360}{159} = 2.26\text{x}$$

With the skill, the same agent scores 2.26 times the points.

My rule of thumb



Then open the evals

Would anyone type this prompt?

Could an agent without the skill pass it at all?

Are the points weighted towards what matters?

Is the task an example the skill already contains?

> **Codex, writing the evals for my streams skill**
>
> I'm not going to cheat the lift.

Minutes later, same run, no new prompt from me:

80 of 100 points for copying this command out of the skill:

```
rg -n "count\\(\\)\\s*>\\s*0|collect\\([^;]+\\)\\.is
Empty\\(|collect\\([^;]+\\)\\.size\\(|sorted\\([^;]*
\\)\\.findFirst\\(|sorted\\(\\)\\.findFirst\\(|filte…
```

From the skill's hard-stops file, java-streams evals, June 2026

An eval that only the skill can pass is not a test.

Three weeks later

ALMOST RIGHT

```
if (workflow.isEmpty()) { return; }
Path path = workflow.get().toAbsolutePath();
```

> **My prompt**
>
> Is that really the best way to build this `Optional`? Check with the `Optional` skill.

```
workflow.ifPresent(this::requireExisting);
```

One prompt later. The skill was fine. It had never triggered.

The capture loop

Two skills in the project fire when I correct `Optional` or stream code

It files an issue in the skill's repository:
before, prompt, after, why

Until the issue closes,
the project carries an override

When the issue closes,
the agent removes the override

One task for my streams skill

> **Summary of the eval's task**
>
> An online shop shows each user their favorite products.
>
> Checking whether a product is in stock is a blocking remote call.
>
> A user can have hundreds of favorites: one by one is too slow.
>
> Use a maximum of 8 checks in parallel for each user.
>
> Return the favorites that are in stock, sorted by name.
>
> Use Java streams.

Without the streams skill

```
static final Semaphore CHECKS = new Semaphore(8);

return user.favoriteProducts().parallelStream()
        .filter(product -> {
            CHECKS.acquireUninterruptibly();
            try { return check(product); }
            finally { CHECKS.release(); }
        })
        .sorted(comparing(Product::name)).toList();
```

All users share one pool of 8 checks.

With the streams skill

```
return user.favoriteProducts().stream()
        .gather(Gatherers.mapConcurrent(8, product ->
                entry(product, check(product))))
        .filter(Map.Entry::getValue)
        .map(Map.Entry::getKey)
        .sorted(comparing(Product::name))
        .toList();
```

Each user gets their own pool of 8 checks.

My own history, one project

1,828 messages I typed
in Symphony for Trello

20 of them corrected
`Optional` or stream code

17 before the project install,
3 after, on the same day

0 in the two weeks after

The fix became the lesson.

The lesson became a skill.

The skill made the mistake stop coming back.

When to create a skill

You corrected the agent twice for the same thing

You copied a paragraph from one `AGENTS.md` into another

You keep a file of prompts to paste

You reuse an old conversation because it already knows what you want


YO DAWG, I HEARD
YOU LIKE SKILLS
SO I MADE A SKILL
THAT BUILDS SKILLS

fm.ht/dm26

Next time an agent is almost right

Get it to do what you wanted,
in the conversation.

Claude Code: `/build-a-skill`
Codex: `$build-a-skill`

It replays your first prompt
with the skill installed.

It writes and runs the evals.
You read them.

Questions?

François Martin

Senior Full Stack Software Engineer

KARAKUN

fmartin.ch

martinfrancois

/in/françoismartin

https://fm.ht/dm26





